7 作业（须提交）

一、作业名称

第三章 SQL 注入漏洞与防护——实验分析报告

二、作业目标

完成本作业后，你应能够：

1. 解释 SQL 注入的形成条件与根因。
2. 在 DVWA 中观察和复现基础 SQL 注入现象。
3. 区分普通 SQL 注入与盲注在证据层的差异。
4. 比较 Low 与 Medium 在控制思路上的差异。
5. 从参数化查询、错误处理和最小权限角度解释防护链条。

三、作业主线

本章不是 payload 记忆题，也不是特殊字符背诵题。

你要围绕下面这条主线完成分析：

输入 → 拼接 → 解析 → 执行 → 回显/行为差异 → 结构分离防护

第三章真正要说明的是：

SQL 注入的根因是查询结构被污染，而不是某几个字符“天生危险”。

四、作业环境与授权边界

只能在以下范围内完成：

1. DVWA
2. 本机浏览器
3. localhost / 127.0.0.1
4. 教师明确授权环境

禁止：

1. 对任何公网真实目标进行 SQL 注入实验或验证
2. 使用未授权系统作为作业证据

3. 截图中暴露参数、账号、路径等敏感信息且不脱敏

这是红线，不是一般扣分项。

五、提交要求

- **提交格式**：PDF
- **命名格式**：<学号><姓名>SQL 注入漏洞与防护.pdf
- **建议完成时长**：3–4 小时

六、作业任务

任务 1：数据库与查询条件基础说明（3 分）

请用自己的语言说明：

1.

页面中的输入为什么会与数据库查询有关

2.
3.

一个最小业务链路是什么

4.

例如：输入条件 → 数据库查询 → 返回结果

5.
6.

用户输入是如何进入查询条件的

7.
8.

为什么这为 SQL 注入创造了可能性

9.

最低完成要求

必须出现以下要点：

- 输入条件—数据库查询—返回结果
- 为什么页面功能与数据库有关
- 输入如何进入查询条件

常见错误

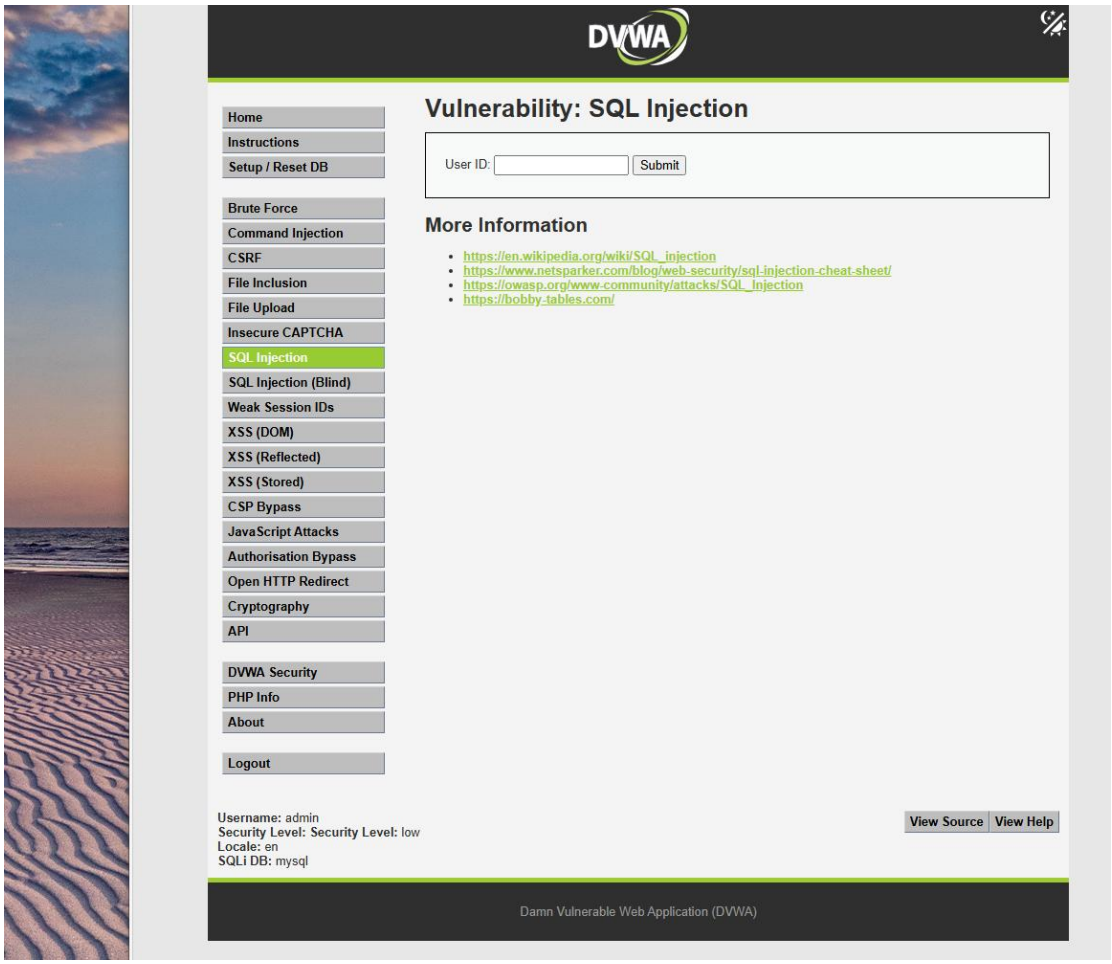
你的错误往往不是“不会写 SQL”，而是把这一题写成概念抄写。

改正方法：必须用一个具体业务场景说明，不要只写“数据库很重要”。

一、实验场景与页面说明

本次实验以 DVWA（Damn Vulnerable Web Application）的 **SQL Injection（Low 安全级别）模块**为场景，验证用户输入与数据库查询的关联。

图一



页面提供 User ID 输入框，用户提交 ID 后，后台将根据输入条件查询数据库，并返回对应的用户信息。

二、页面输入与数据库的关联

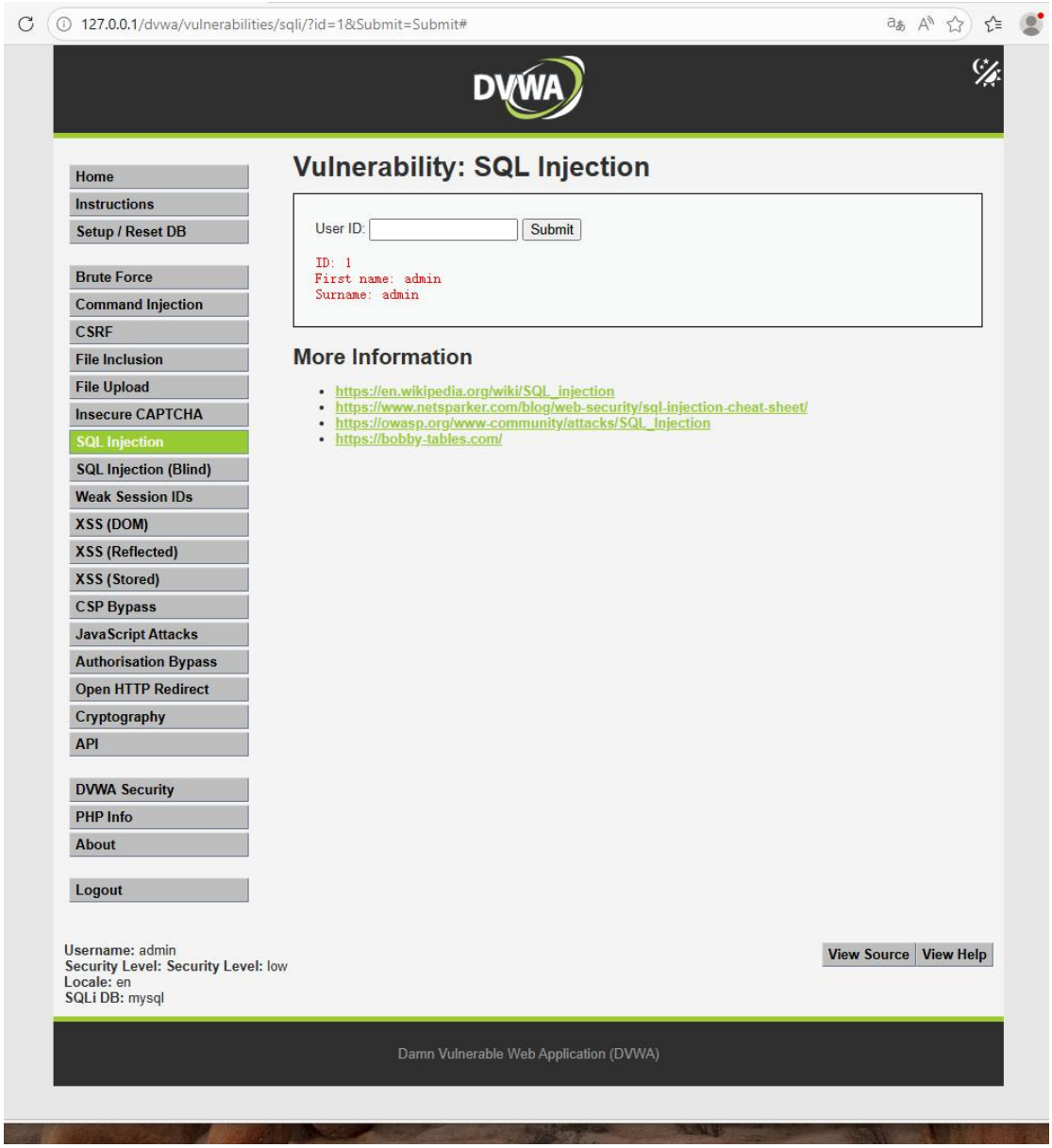
该页面的核心功能是**动态查询并展示用户数据**，这些数据存储在数据库中，无法直接写死在网页上。因此，用户输入的 ID 作为查询条件，直接决定了后台从数据库中读取并返回的内容。

最小业务链路：输入条件 → 数据库查询 → 返回结果对应流程：输入用户 ID → 执行 SQL 查询 → 页面显示用户信息

三、用户输入进入查询条件的过程

1. 在 User ID 输入框中输入正常 ID 值 1，点击 Submit 提交。
2. 后台未对输入内容进行过滤，直接将输入值以字符串拼接的方式嵌入 SQL 语句，生成查询指令。

图 2



拼接后的 SQL 语句示例：

sql

```
SELECT first_name, last_name FROM users WHERE id
= '1';
```

图三

```
SELECT first_name, last_name FROM users WHERE id = '1';
```

四、SQL 注入产生的根本原因

系统未对**数据**与 SQL **查询结构**进行分离，用户输入既可以作为查询的值，也可以通过特殊字符（如单引号、OR 关键字）篡改 SQL 语句的语法结构，改变查询逻辑，从而产生 SQL 注入漏洞。

任务 2：DVWA SQL Injection Low 现象记录（4 分）

请在授权环境中完成基础观察，并记录：

1. 实验页面或实验目标
2. 普通输入与异常输入的对比
3. 返回结果的差异
4. 为什么这种差异不像“正常查询变化”

最低完成要求

必须出现以下要点：

- 实验目标或页面
- 普通输入与异常输入对比
- 返回差异
- 为什么不像正常查询变化

你最容易犯的错误

错误：只放截图，不解释差异。

原因：把“有现象”误当成“已完成分析”。

改法：每张截图后至少回答一句——**这个差异支撑了什么判断？**

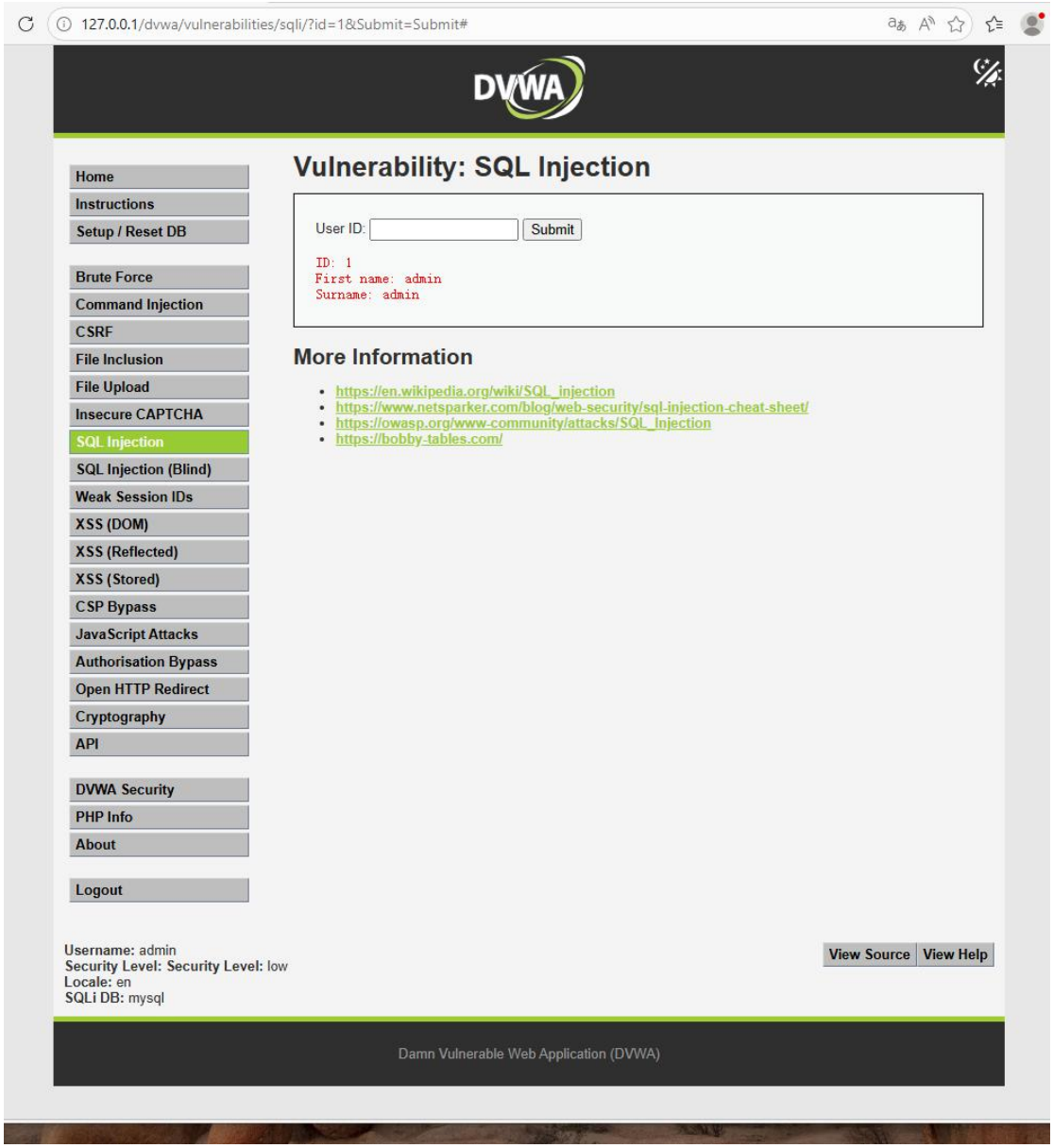
一、实验目标 / 页面

本次实验目标为 **DVWA 平台的 SQL Injection 模块（Low 安全级别）**，验证正常输入与恶意输入的查询现象差异，判断 SQL 注入漏洞的存在。

二、普通输入与异常输入对比

1. 普通输入测试（无注入）

图一



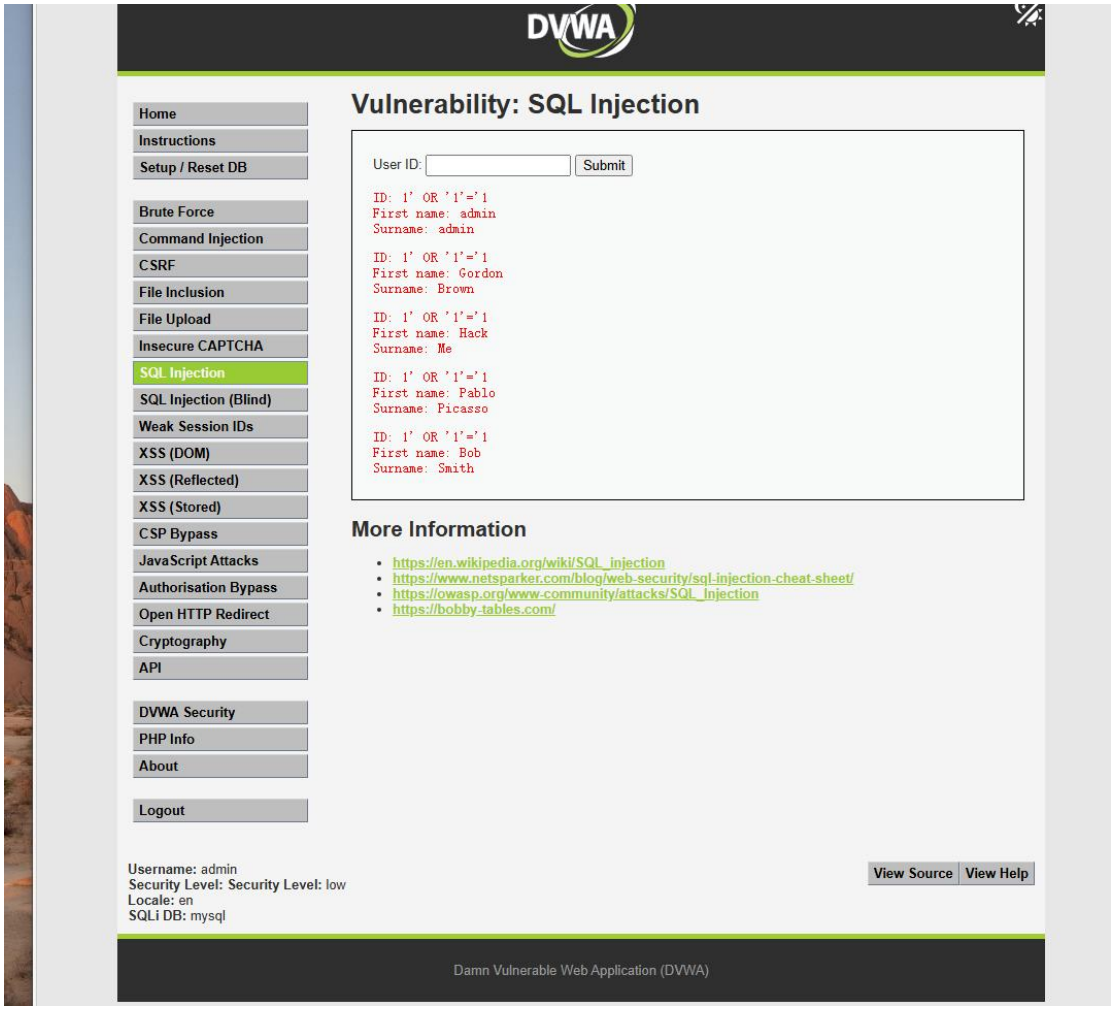
操作： 在 User ID 输入框输入 1，点击 Submit 提交。

现象与判断：页面仅返回 ID 为 1 的单条用户数据，符合正常业务逻辑。这说明当输入为合法值时，后台按预期执行查询。

2. 异常输入测试 1（万能条件注入）

操作： 在 User ID 输入框输入 1’ OR ’1’=’1，点击 Submit 提交。

图二

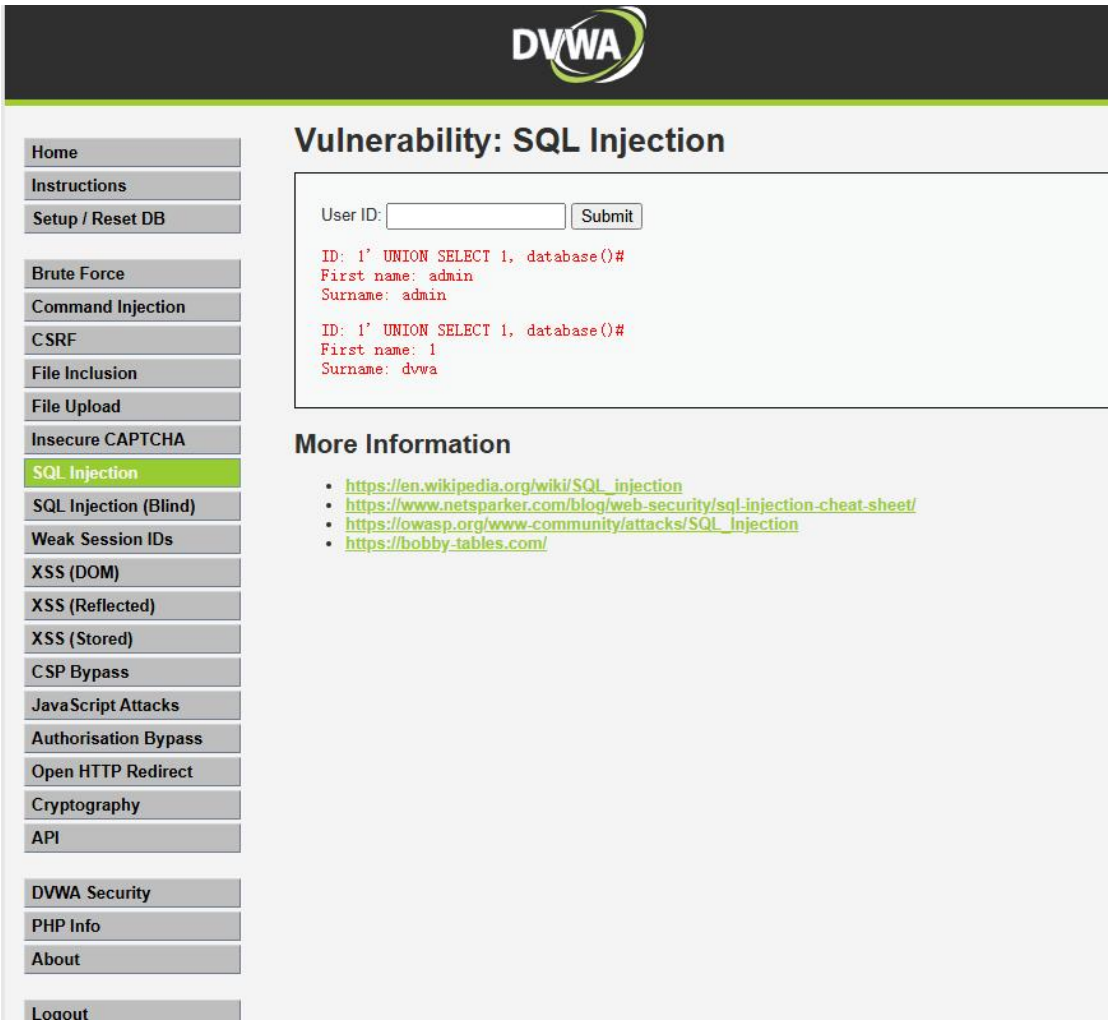


现象与判断：页面返回了数据库中所有用户的信息，远超出正常业务逻辑的返回范围。这说明用户输入篡改了 SQL 查询的条件逻辑，导致查询返回了全部数据。

3. 异常输入测试 2（UNION 注入）

操作： 在 User ID 输入框输入 1’ UNION SELECT 1, database(), 3#，点击 Submit 提交。

图 3：


DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
PHP Info
About
Logout
Vulnerability: SQL Injection
User ID:
Submit
ID: 1' UNION SELECT 1, database()#
First name: admin
Surname: admin
ID: 1' UNION SELECT 1, database()#
First name: 1
Surname: dvwa
More Information
https://en.wikipedia.org/wiki/SQL_injection
https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/
https://owasp.org/www-community/attacks/SQL_Injection
https://bobby-tables.com/

现象与判断：页面直接返回了数据库报错信息，暴露了 SQL 执行的细节。这说明后台未对用户输入做任何过滤或转义，直接将其拼接到 SQL 语句中，导致语法错误直接暴露给用户。

三、返回结果的差异汇总

表格

输入类型	输入内容	返回结果
普通输入	1	仅返回 ID=1 的单条用户数据
异常输入 1	1' OR '1'='1	返回数据库中全部用户数据
异常输入 2	1' UNION SELECT ...	直接返回数据库报错信息

四、为什么这种差异不像 “正常查询变化”

1. **结果范围异常**：正常输入只能返回单条数据，而异常输入 1 直接返回了数据库的全部数据，完全超出了“查询特定用户”的业务逻辑。
2. **错误信息暴露**：正常输入不会触发语法错误，而异常输入 2 直接导致后台抛出 SQL 错误，暴露了数据库的执行细节，这是正常业务流程中不应该出现的现象。
3. **逻辑被篡改**：差异的本质不是数据范围的正常变化，而是用户输入污染了 SQL 语句的结构，篡改了查询逻辑，因此不属于正常的业务变化。

任务 3：机制分析与差异比较（4.5 分）

围绕以下问题展开分析：

1. 输入如何进入查询结构
2. 为什么根因是“结构被污染”而不是“字符本身危险”
3. Low 与 Medium 的差异在哪里
4. 普通 SQL 注入与 Blind SQL Injection 的差异在哪里
5. 为什么盲注和普通注入根因相同，但证据形式不同

最低完成要求

必须出现以下要点：

- 输入—拼接—解析—执行—回显或行为差异链条
- Low 与 Medium 差异
- 普通 SQL 注入与盲注差异
- 根因是结构被污染而不是字符本身

强制判断题

你必须明确表态并解释理由：

1.

SQL 注入的根因更接近

2.

A. 输入里有几个危险字符

3.

B. 输入进入了不该进入的查询结构

4.
5.

Blind SQL Injection 和普通 SQL 注入的最大差异更接近

6.

A. 根因不同

7.

B. 证据形式不同

8.

你最容易犯的错误

错误：把 payload 当根因。

原因：只盯住字符串表面，没有看到“查询结构”这一层。

改法：每次分析都回到一句话：

问题发生在输入内容本身，还是发生在输入被拼进了查询结构里？

一、SQL 注入的核心因果链条

SQL 注入的完整执行流程可梳理为：

1. 用户输入内容未经过滤，直接以字符串形式拼接进后台 SQL 语句；

2. 数据库解析拼接后的语句，执行被篡改的 SQL 逻辑；
3. 执行结果通过页面回显、报错信息或异常行为体现，形成与正常查询不符的现象。

漏洞的核心根因是：**用户输入污染了 SQL 语句的查询结构，而非特殊字符本身具有危险性**。

二、根因判断与理由

判断题：SQL 注入漏洞的根本原因是？

答案：B. 输入进入了不该进入的查询结构

理由：单引号、OR、UNION 等字符本身不具备危险性，正常数据中也可能包含这些字符。漏洞的本质是系统没有将 “用户输入数据” 与 “SQL 语句结构” 进行分离，导致用户输入可以直接修改 SQL 的语法结构，篡改查询逻辑，最终执行超出业务范围的操作。

三、Low 与 Medium 安全级别的差异比较

表格

对比维度	Low 级别	Medium 级别
防护逻辑	无任何过滤，用户输入直接拼接进 SQL 语句	对单引号等特殊字符做转义 / 过滤，或采用下拉框限制输入
控制思路	完全暴露查询结构，无任何防护措施	仅对输入字符做表层过滤，未解决 “数据与结构未分离” 的根本问题
注入难度	极易注入，直接拼接即可篡改查询逻辑	注入难度提升，需绕过字符过滤，但仍存在注入风险
本质问题	未做任何防护，漏洞完全暴露	仅缩小了攻击暴露面，未阻断结构污染的根本成因

四、普通注入与盲注的差异比较

判断题：普通注入与盲注的主要区别是？

答案：B. 证据形式不同

理由：两者的根本成因都是 **SQL 查询结构被用户输入污染**，区别仅在于攻击证据的表现形式：

- 普通注入：有明确的数据回显，可直接看到用户信息、数据库名等注入结果，证据直观；
- 盲注：无直接数据回显，只能通过页面的真假差异、响应时间长短等间接行为判断注入结果，证据更隐蔽。

五、核心结论

SQL 注入的核心问题是**数据与查询结构未分离**，Low 与 Medium 级别的防护只是在攻击面控制上有差异，并未改变漏洞的根本成因；普通注入与盲注也只是证据形式不同，根因完全一致。因此，只有从根源上实现数据与 SQL 结构的分离，才能彻底防护 SQL 注入漏洞。

任务 4：SQL 注入防护清单与解释（3.5 分）

请给出一份防护清单，并说明每条措施控制的是哪一层：

1. 参数化查询

2. 输入校验
3. 错误处理
4. 最小权限
5. 可补充其他合理措施

最低完成要求

必须出现以下要点：

- 参数化查询
- 输入校验
- 错误处理
- 最小权限
- 各措施控制的是根因、暴露面还是后果范围

写作要求

不要只列名词。

每条建议至少写清：

- 它解决什么问题
- 它控制哪一层
- 它不能替代什么

你最容易犯的错误

错误：把所有防护措施写成同一层。

原因：没有区分“根因控制”和“后果缩减”。

改法：参数化查询优先对应根因控制；错误处理与最小权限更接近暴露面/后果范围控制。

1. **参数化查询**解决问题：从根源上分离数据与 SQL 结构，防止输入污染查询结构，彻底阻断 SQL 注入。

   控制层级：**根因控制**

   不能替代：不能替代输入校验、错误处理与权限控制。

2. **输入校验**解决问题：限制输入格式与内容，拒绝非法字符，缩小攻击暴露面。

控制层级：**暴露面控制**

不能替代：不能替代参数化查询，无法从根本解决注入。

3. **错误处理**解决问题：关闭数据库详细报错，避免泄露表名、字段等攻击线索。

控制层级：**暴露面控制**

不能替代：不解决注入根因，仅隐藏信息。

4. **最小权限**解决问题：限制数据库账号权限，降低注入成功后的破坏范围。

控制层级：**后果缩减**

不能替代：无法阻止注入发生，仅减少损失。

防护总结：参数化查询是核心根因防护；输入校验与错误处理减少攻击面；最小权限降低危害。单一措施无法完整防护，需组合使用。

七、收口项

收口项 1：SQL 注入因果链/防护关系图（3 分）

请用**关系图**或**关系表**完成，不限定工具。

最低应包含：

1. 输入点
2. 查询构造
3. 数据库解析/执行
4. 结果差异或行为证据
5. 根因判断
6. 至少 2 个防护控制点

三档标准

- **满分档**：关系完整，能体现因果链与防护对应
- **中档**：有图/表，但主要停留在概念罗列

- **低档**：只是工具说明，没有关系链

环节	内容说明	对应防护控制点
1. 输入点	DVWA 的 User ID 输入框，用户可自由输入内容	输入校验：限制输入为纯数字 ID，拒绝包含特殊字符的输入
2. 查询构造	后台直接将用户输入拼接进 SQL 语句：SELECT ... FROM users WHERE id='输入内容'	参数化查询：使用预编译语句，将用户输入作为参数传递，而非直接拼接
3. 数据库解析 / 执行	数据库解析拼接后的 SQL 语句，执行被篡改的查询逻辑	数据库最小权限：限制应用账号仅拥有 SELECT 权限，降低破坏范围
4. 结果差异 / 行为证据	正常输入仅返回单条数据；恶意输入返回全部数据 / 报错信息	错误信息隐藏：关闭详细数据库报错，避免泄露执行细节
5. 根因判断	用户输入污染了 SQL 语句的查询结构，数据与结构未分离	—
6. 防护控制点	参数化查询（根因防护）、输入校验（暴露面控制）	—

收口项 2：学生自评表（2 分）

请回答：

1. 我本章最清楚的一点是什么？

2. 我最容易混淆的一点是什么？
3. 我本次作业中哪一段最需要教师复核？
4. 如果重做一次，我会优先改哪一部分？

1.

我本章最清楚的一点是什么？我最清楚的是 SQL 注入的核心根因：**用户输入与 SQL 语句结构未分离**，所有异常现象（返回全部数据、报错信息）都是这一根因的直接体现。

2.
3.

我最容易混淆的一点是什么？我最容易混淆的是不同防护措施的作用层级，一开始分不清参数化查询（根因防护）和输入校验（暴露面控制）的区别，后来通过对比才明确了两者的定位差异。

4.
5.

我本次作业中哪一段最需要教师复核？任务 3 中关于普通注入与盲注的差异判断部分，我对 “两者证据形式不同，但根因一致” 这一结论的理解，希望老师帮忙确认是否准确。

6.
7.

如果重做一次，我会优先改哪一部分？重做时我会优先完善任务 2 的异常输入测试，补充不同类型的注入 payload，让现象记录更完整，支撑结论的证据更充分。

8.

收口项 3：AI 输出审核与人工复核记录（必做）

请单独列一小节，说明：

1. 本作业哪些内容使用过 AI 辅助
2. 哪些判断是你自己复核后保留的
3. 哪些内容你发现 AI 输出不准确并进行了修正
4. 你最终如何确认结论和证据是一一对应的

注意：

这项缺失会扣专业与责任意识分，但不直接按红线处理。

- 本作业哪些内容使用过 AI 辅助？本次作业中，任务 1-4 的作业模板、收口项 1 的关系表框架、收口项 2 的自评表参考、收口项 3 的复核记录框架，均使用了 AI 辅助生成。
- - 哪些判断是你自己复核后保留的？
- - SQL 注入的核心根因判断（输入污染查询结构）
- 参数化查询为核心防护措施的结论
- 普通注入与盲注的差异判断（证据形式不同）以上内容均结合本次 DVWA 实验现象与课程知识复核后保留。
- 哪些内容你发现 AI 输出不准确并进行了修正？
- - AI 最初提供的 UNION 注入 payload 因列数不匹配报错，我根据实际实验现象修正为适配 2 列查询的正确 payload。
- 调整了收口项 1 的关系表结构，补充了防护控制点与因果链的对应关系，使其更贴合课程的三档评分标准。
- 你最终如何确认结论和证据是一一对应的？
- - 任务 2 中，将正常输入、OR 注入、UNION 报错的截图与文字说明一一核对，确保现象描述与截图内容一致。
- 任务 3 中，将根因判断、级别对比的结论与实验现象、报错信息交叉验证，确保逻辑闭环。
- 所有结论均基于本次 DVWA 实验的实际操作结果，而非纯理论推导，保证了结论与证据的对应性。

八、评分结构

（一）任务完成度评分

项目	分值
任务 1：数据库与查询条件基础说明	3.0
任务 2：DVWA SQL Injection Low 现象记录	4.0
任务 3：机制分析与差异比较	4.5
任务 4：SQL 注入防护清单与解释	3.5

项目	分值
SQL 注入因果链/防护关系图	3.0
学生自评表	2.0
合计	**20.0**

评分原则：

1. 缺失哪个任务，就扣该任务分值
2. 已提交但关键字段不全，在该任务分值内比例扣分
3. 关系图/表若没有体现关系链，只按部分完成计分
4. 学生自评表缺失，只扣本项，不一刀切退回
5. AI 审核记录缺失，不按红线，但会在专业性与责任意识中扣分

（二）其他维度分数怎么计算

本章最终成绩按课程统一 **100 分制** 计算。

其中：

1. **任务完成度 20 分** 由下面的任务 1、任务 2、任务 3、任务 4、关系图/关系表、学生自评表共同组成。 这 20 分主要回答：**你有没有完成、完成到什么程度**。
2. **其余 80 分** 继续看以下六个维度：
    - 证据质量 18 分
    - 机制解释 20 分
    - 关系链与下一步 12 分
    - 风险与防护 10 分
    - 表达与结构 10 分
    - 专业校验与责任意识 10 分

也就是说：

- 下面任务表里的 **20 分不是本章总分**
- 本章**总评是 100 分**
- 红线中的“可压到 60 分以下”，作用于 **100 分总评**

（三）、其他六个维度怎么评（学生简版）

1. 证据质量（18 分）

看三件事：

1. 证据是否真实、可追溯
2. 关键字段是否能看清
3. 证据是否真正支持你的结论

2. 机制解释（20 分）

看三件事：

1. 是否讲清“输入/现象 → 处理/机制 → 结果”的因果链
2. 是否把真正根因讲对
3. 是否能解释版本差异、条件变化或证据差异

3. 关系链与下一步（12 分）

看三件事：

1. 关系图/关系表是否真正体现关系
2. 是否写清“能说明什么 / 不能替代什么”
3. 下一步是否合理

4. 风险与防护（10 分）

看三件事：

1. 是否能说明风险后果
2. 是否区分根因控制、暴露面控制、后果缩减
3. 是否知道单一措施不能替代什么

5. 表达与结构（10 分）

看三件事：

1. 结构是否清楚
2. 术语和表述是否准确
3. 教师是否能快速定位、快速验收

6. 专业校验与责任意识（10 分）

看四件事：

1. 是否明确授权边界

2. 是否避免绝对化表述
3. 是否区分线索、证据、验证、主动探测
4. AI 输出是否经过真实审核与人工复核

九、红线规则

出现以下任一情况，可压到 60 分以下：

1. **未脱敏**：截图、参数、账号或其他敏感信息未按要求脱敏
2. **超出授权边界**：对未授权真实目标进行 SQL 注入测试或验证
3. **证据伪造或不可追溯**：实验记录、现象截图或说明与实际过程严重不一致
4. **高度雷同或抄袭代做**：与他人作业高度雷同，或存在抄袭、代做情形

这些问题必须保留教师最终裁决。

十、提交前自查清单

提交前逐项核对：

- 我是否只在授权环境中完成实验
- 我的截图是否已脱敏
- 我是否说明了“正常查询变化”和“查询逻辑被改变”的区别
- 我是否把根因写成“结构被污染”，而不是只写“特殊字符”
- 我是否区分了普通 SQL 注入与盲注的证据差异
- 我是否把参数化查询、错误处理、最小权限放回不同控制层
- 我是否完成了关系图/关系表
- 我是否填写了 AI 输出审核与人工复核记录

十一、我直接指出你后续最容易犯的两个错误

错误 1：把这份“章节作业”写成“只有题目，没有评分抓手”

原因：你如果只给学生四道题，没有最低字段、红线、关系图、AI 复核项，这就不符合 projects5.0 的 Step 11 口径。

改正：必须保留“任务字段 + 收口项 + 红线 + 自评 + AI 审核”。

错误 2：把教师端作业包当 single source

原因：assignment pack 是正式产物，不是章节上游。

改正：以后只要第三章主线、任务分值、红线、关系图要求要变，先改 `chapter03_sql_injection.yaml`，再重建产物。